REGRET IN RUNTIME ANALYSIS OF BLACK-BOX ALGORITHMS

A report submitted to The University of Manchester
for the degree of Bachelor of Science
in the Faculty of Science and Engineering

2026

Manoj Manikandan

Supervised by Dr. Jonathan Shapiro

Department of Computer Science

Contents

Abstract **6**

Declaration **7**

Acknowledgements **8**

1 Introduction **10**

1.1 Motivation 10

1.1.1 Conventional Solutions 10

1.1.2 The Problem 11

1.1.3 Project's Alternative 11

1.2 Objectives 12

1.2.1 General Aims 12

1.2.2 Theoretical Exploration 12

1.2.3 Supplementary Software Development 12

1.2.4 Experimental Analysis 12

1.3 Project Outline 13

1.3.1 Literature Study 13

1.3.2 Isolating Relevant Benchmarks 13

1.3.3 Development of Experiment framework 13

1.3.4 Orchestrating Experiments 13

1.3.5 Evaluating Results 13

1.3.6 13

1.4 Ethical Considerations 14

1.5 Report Structure 15

1.5.1 Background 15

1.5.2 Software Design 15

1.5.3 Experiment Structure 15

1.5.4 Evaluation & Discussion of Results 15
1.5.5 Conclusion & Future Work 15

2 Background **16**
2.1 Computational Problems 16
2.1.1 Problem Formulations 16
2.1.2 Algorithms 17
2.1.2.1 Algorithm Properties 17
2.1.2.2 Oracle-Based Computation 17
2.1.3 Search Space 17
2.1.3.1 Structure of the Search Space 17
2.1.3.2 Constraints and Feasibility 18
2.1.4 Objective Functions and Landscape Properties 18
2.1.4.1 Zeroth-, First-, and Second-Order Methods . . . 18
2.1.4.2 Landscape Properties 19
2.1.5 Complexity Measures 20
2.1.5.1 Classical Complexity: Time and Space 20
2.1.5.2 Information Complexity: Query and Oracle Complexity 20
2.1.5.3 Expected vs. Worst-Case Analysis 20
2.1.6 Probabilistic Tools for Algorithm Analysis 20
2.1.6.1 Expectation and Variance 20
2.1.6.2 Concentration Inequalities 20
2.2 Black-Box Optimization 21
2.2.1 The Black-Box Model 21
2.2.1.1 Variants of Black-Box Models 21
2.2.2 White-Box vs. Black-Box Optimization 21
2.2.3 Black-Box Complexity 21
2.2.4 Purpose of Randomized Search Heuristics 21
2.2.4.1 No-Free-Lunch Theorems 21
2.2.4.2 Taxonomy of Derivative-Free Methods 21
2.2.5 Randomized Search Heuristic Algorithms 21
2.2.5.1 Evolutionary Algorithms (EA) 21
2.2.5.2 Simulated Annealing 21
2.2.5.3 Randomized Local Search (RLS) 21
2.2.6 Few Pseudo-Boolean Problems 22

2.2.6.1 OneMax 22
2.2.6.2 LeadingOnes 22
2.2.6.3 Jump 22
2.2.6.4 BinaryValue[BinVal] 23
2.2.6.5 Trap 23
2.2.6.6 Plateau 23
2.2.6.7 Hierarchical If-Only-If 23
2.2.6.8 MaxSAT 23
2.2.6.9 Tunable Landscapes: NK-Landscapes 24
2.2.7 Traditional Performance Measures in BBO 24
2.3 Sequential and Online Optimization Frameworks 25
2.3.1 Exploration-Exploitation Trade-off 25
2.3.2 Multi-Armed Bandits 25
2.3.3 Online Optimization 25
2.3.4 Reinforcement Learning 25
2.4 Computational Regret 26
2.4.1 Definition of Regret 26
2.4.2 Types of Regret 26
2.4.2.1 Instantaneous Regret 26
2.4.2.2 Cumulative Regret 26
2.4.2.3 Simple Regret 27
2.4.2.4 Normalized Regret 28
2.4.3 Regret in Black-Box Optimization 28
2.4.4 Regret Bounds 28
2.4.5 Connections to Runtime Analysis 28

3 Supplementary Software Design 29
3.1 Purpose of the Software 29
3.1.1 Preexisting software 29
3.1.2 Capabilities 30
3.1.3 Design Decisions 31
3.1.3.1 Dispatching framework 31
3.1.3.2 UML Diagram 31
3.1.4 Limitations 32
3.1.5 Result Interpretations 34
3.2 Testing strategy 36

3.3 Project Dependencies 37

4 Experiments **38**
4.1 Experiment Structure 38
4.1.1 Repetitions and budget 38
4.1.2 Execution mode 38
4.1.3 Visualisation and Table Strategy 39
4.1.4 Experiment numbering 41
4.2 Runtime Profiles 41
4.2.1 Motivation 41
4.2.2 Fixed-Budget Analysis 42
4.2.3 Variants 43
4.2.3.1 Expectation-Based Runtime Profiles 43
4.2.3.2 Distributional Runtime Profiles 43
4.2.4 Connection to Cumulative Regret 44
4.2.5 Practical Relevance 47
4.3 Baseline Suites 48
4.3.1 Suite 01: Algorithm-Family Comparison 48
4.3.2 Suite 02: Extended Pseudo-Boolean Benchmark 49
4.4 Exploratory Analysis of Regret and Runtime 49

5 Evaluation & Discussion of Results **50**
5.1 PLACEHOLDER 50
5.2 PLACEHOLDER 50
5.3 PLACEHOLDER 50

6 Conclusion & Future Work **51**
6.1 Conclusion 51
6.2 Future Work 51

Word Count: 6266

Abstract

TODO: Add abstract after completion of report

Declaration

No portion of the work referred to in this report has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Jonathan Shapiro, for his disproportionate support and guidance throughout this project. I also thank my second marker for taking their time to evaluate and assess my work. Moreover, Moreover, I am immensely grateful to my family and friends for their immeasurable support and for encouraging my ambitious pursuits throughout my degree.

Chapter 1

Introduction

1.1 Motivation

Many aspects of our day-to-day lives revolves around finding optimality in problems that we can have no comprehensible insight about. It can be observed in industrial processes of ! TODO!. These types of problems are said to be Black-Box Optimization (BBO) problems.

We do not get much tools to interact with them save from a cost/objective function $\mathbf{f} : \mathcal{X} \mapsto \mathcal{S}$ (the 'oracle') that provides an abstract representation of the problem, where $\mathcal{X}$ is the search space, $\mathcal{S}$ the evaluation space !TODO: find concrete refs!. Typically, $\mathcal{S} \subseteq R$. The task is to minimize or maximize the solutions produced by this objective function, i.e., find a solution that Much study is done in optimizing this cost function without the liberty of classic mathematical tools such as derivability, smoothness and convexity (these methods warrant the oracle to return additional information, such as gradients, apart from the evaluation value of an input [28]).

1.1.1 Conventional Solutions

Classical analysis of solving BBO problems are concerned with the so-called Black-Box Complexity (BBC), that is, deriving the runtime bounds for an algorithm to sample a (global) optimum, depending only on the problem size $\mathbf{n}$. The reasoning behind this is palpable; the *computational* complexity of evaluating the objective function for such BBO systems are usually expensive.

An obvious example is Hyperparameter Tuning in Machine Learning (ML)

models !CITE!. The optimal hyperparameters could be thought of as the *optima* for the Black-Box Optimization Problem (BBOP) of the tuning that maximizes their validation loss [1].

1.1.2 The Problem

While this approach secures the bounds on the first sampling of the optimum, it presents no clues on the solution quality produced by the algorithm. Consider the case when the algorithm samples the optimum at time t, then in the next step $t+1$, moves completely away from the vicinity. This algorithm might then produce worse answers, asymptotically.

1.1.3 Project's Alternative

[1] 'how well the model fits the underlying probability distribution of the data inputs & outputs'

1.2 Objectives

1.2.1 General Aims

1.2.2 Theoretical Exploration

1.2.3 Supplementary Software Development

1.2.4 Experimental Analysis

1.3 Project Outline

1.3.1 Literature Study

1.3.2 Isolating Relevant Benchmarks

1.3.3 Development of Experiment framework

1.3.4 Orchestrating Experiments

1.3.5 Evaluating Results

1.3.6

1.4 Ethical Considerations

1.5 Report Structure

1.5.1 Background

1.5.2 Software Design

1.5.3 Experiment Structure

1.5.4 Evaluation & Discussion of Results

1.5.5 Conclusion & Future Work

Chapter 2

Background

2.1 Computational Problems

2.1.1 Problem Formulations

A *computational problem* is a formal specification of a task defined over a set of *problem instances*. Each instance encodes the input data relevant to a particular occurrence of the problem. The *problem class* describes the collection of all such problem instances together with the corresponding output requirements. [20]

Problems are broadly categorized by the nature of their required output.

- A *decision problem* demands a binary answer, whether a given instance satisfies some property.
- A *search problem* asks for the construction of an explicit solution satisfying a specified property, such as a feasible assignment or a path in a graph.
- An *optimization problem* further requires that the returned solution be optimal (or near-optimal) with respect to some objective function. [20]

A further distinction concerns the role of randomness in the problem formulation. In a *deterministic* formulation, both the instance and every algorithm operating on it are fully specified without any probabilistic component. In a *stochastic* formulation, the instance itself, the objective evaluations, or the algorithm may involve randomness. Noisy objective functions, where repeated evaluation of the same candidate solution may return different values, are a canonical example and arise naturally in simulation-based and real-world settings.

2.1.2 Algorithms

2.1.2.1 Algorithm Properties

Deterministic and Randomized Algorithms.

Adaptive vs. Non-Adaptive Algorithms

2.1.2.2 Oracle-Based Computation

An *oracle* is an abstract computational device that answers queries about a problem instance. In optimization, the most common oracle is the *function evaluation oracle*, which, given a query point $x \in \mathcal{X}$, returns the objective value $f(x)$ (or a noisy estimate thereof). The algorithm interacts with the oracle in a *query-response* model: it submits a query and receives a response before deciding on the next query. The total number of such interactions constitutes the *query complexity* of the algorithm, formalized in Section 2.1.5.2 [30; 12].

2.1.3 Search Space

2.1.3.1 Structure of the Search Space

The *search space* $\mathcal{X}$ is the set of all candidate solutions considered by an algorithm. Search spaces may be *finite* or *infinite*, and *discrete* or *continuous*.

Discrete combinatorial search spaces, in particular the Boolean hypercube $\{0,1\}^n$, are central to the theoretical analysis of evolutionary algorithms (EA) and other randomized search heuristics [11; 21] used in theoretical BBO analysis. Continuous spaces $\mathbb{R}^d$ arise in numerical optimization [30], and mixed spaces containing both discrete and continuous variables occur in practice but are considerably harder to analyze [4].

The structure of the search space, and in particular how neighboring solutions relate to one another, fundamentally influences the difficulty of optimization. This is encoded by a *neighborhood function* $\mathcal{N} : \mathcal{X} \to 2^{\mathcal{X}}$, assigning to each candidate solution the set of solutions reachable in a single step!CITE: Hoos and Stützle - 2005 - INTRODUCTION!. For the Boolean hypercube, the standard neighborhood consists of all strings at Hamming distance one !CITE: a resource using hamming distance!.

Remark 1 (Problem domain of interest)**.** *This project is concerned with* Pseudo-Boolean, Combinatorial Optimization problems*, with solution space* $\mathcal{X} \subseteq \{0,1\}^n$*. A candidate solution can be represented as a* bitstring*: a sequence of alphabets* $a \in \{0,1\}$*. The objective functions considered for such problems are of the form* $f : \mathcal{X} \to \mathbb{R}_0^+$*, where* $\mathbb{R}_0^+$ *denotes the non-negative reals. Without loss of generality, an objective function with co-domain* $\mathcal{T} \subseteq \mathbb{R}$ *can be mapped to this convention via the affine shift*

$$g : \mathbb{R} \to \mathbb{R}_0^+, \qquad g(x) = x - \min_{y \in \mathcal{X}} f(y),$$

which is well-defined whenever f *attains its minimum on* $\mathcal{X}$*, and preserves the ordering of all objective values by construction.*

2.1.3.2 Constraints and Feasibility

In a *constrained* optimization problem, the *feasible region* $\mathcal{F} \subseteq \mathcal{X}$ is defined by one or more constraints that a valid solution must satisfy [30; 6]. An *unconstrained* problem places no restriction beyond membership in $\mathcal{X}$. Constraints may be *hard* (a solution violating them is inadmissible) or *soft* (violations are penalized in the objective). The boundary of the feasible region can significantly complicate optimization, particularly for local search methods that may generate infeasible intermediate solutions.

2.1.4 Objective Functions and Landscape Properties

The geometric structure of the objective function determines both which algorithms are applicable and what convergence guarantees are attainable. A central organizing principle is that access to higher-order analytical information about f, i.e., gradients, curvature, or structural properties such as convexity, generally enables stronger theoretical guarantees and more efficient algorithms; restricting this access, as in the black-box setting, correspondingly limits what can be guaranteed.

2.1.4.1 Zeroth-, First-, and Second-Order Methods

Optimization algorithms can be classified by the order of analytical information they require from f. This *information hierarchy* has three tiers.

Zeroth-order methods use only evaluations of $f(x)$, with no gradient or curvature information. The algorithm observes only the output of the function evaluation oracle at each queried point and must infer search directions from comparisons of objective values alone. This exactly describes the black-box model formalized in Section 2.2: when the internal structure of f is inaccessible or analytically intractable, zeroth-order methods are the only option. Their weaker information access is the fundamental reason black-box optimization is harder in general than white-box optimization, and it is the regime in which all algorithms studied in this project operate.

First-order methods additionally access the gradient $\nabla f(x) \in \mathbb{R}^d$, which encodes the direction of steepest ascent at x. For a differentiable function $f : \mathbb{R}^d \to \mathbb{R}$, gradient-based methods exploit this to define descent directions and form the backbone of classical continuous optimization [30]. When f is convex but non-differentiable, the gradient generalizes to the *subdifferential* $\partial f(x)$, the set of all *subgradients* g satisfying

$$f(y) \geq f(x) + g^\top (y - x) \quad \forall y \in \mathbb{R}^d.$$

Subgradient methods replace the gradient with any element of $\partial f(x)$, recovering convergence under convexity alone, albeit at rates slower than those attainable with smooth gradients [6].

Second-order methods further access the Hessian $\nabla^2 f(x) \in \mathbb{R}^{d \times d}$, which captures local curvature. Methods that utilize or approximate the Hessian, such as Newton and quasi-Newton schemes, can achieve superlinear or quadratic convergence near a solution [30], at the cost of maintaining a $d \times d$ matrix.

Together, these three tiers define distinct levels of oracle access, and their availability directly governs the choice and efficiency of applicable algorithms. The move from second-order to zeroth-order corresponds to progressively weaker assumptions about what the oracle reveals, culminating in the black-box model where only function values are observable.

2.1.4.2 Landscape Properties

The following geometric properties of f characterize which tier of analytical information is both available and useful.

Convexity. A function $f : \mathbb{R}^d \to \mathbb{R}$ is *convex* if for all $x, y \in \mathbb{R}^d$ and $\lambda \in [0,1]$ [6],

$$f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda) f(y).$$

Convexity guarantees that every local minimum is global, rendering the landscape tractable for first-order methods. *Strong convexity* additionally requires

$$f(y) \geq f(x) + \nabla f(x)^\top (y-x) + \tfrac{\mu}{2}\|y-x\|^2$$

for some $\mu > 0$, implying a unique global minimum and linear convergence rates for gradient descent !CITE: Nesterov 2004!.

Smoothness and Lipschitz Continuity.

Ruggedness and Multimodality.

2.1.5 Complexity Measures

2.1.5.1 Classical Complexity: Time and Space

2.1.5.2 Information Complexity: Query and Oracle Complexity

2.1.5.3 Expected vs. Worst-Case Analysis

2.1.6 Probabilistic Tools for Algorithm Analysis

2.1.6.1 Expectation and Variance

2.1.6.2 Concentration Inequalities

Markov's Inequality.

Chebyshev's Inequality.

Chernoff Bounds.

2.2 Black-Box Optimization

2.2.1 The Black-Box Model

In the *black-box optimization* model, the algorithm has no access to the analytical form of the objective function. It interacts with the objective exclusively through an oracle that, upon receiving a query $x \in \mathcal{X}$, returns $f(x)$ (or a noisy estimate thereof) [12; 10; 27]. This constitutes *zeroth-order* information: no gradient or higher-order information is available. The black-box model is appropriate whenever the objective arises from a simulation, physical experiment, or any process whose internal structure is inaccessible or too complex to exploit analytically.

2.2.1.1 Variants of Black-Box Models

2.2.2 White-Box vs. Black-Box Optimization

2.2.3 Black-Box Complexity

2.2.4 Purpose of Randomized Search Heuristics

2.2.4.1 No-Free-Lunch Theorems

2.2.4.2 Taxonomy of Derivative-Free Methods

2.2.5 Randomized Search Heuristic Algorithms

2.2.5.1 Evolutionary Algorithms (EA)

2.2.5.2 Simulated Annealing

Cooling Schedules

2.2.5.3 Randomized Local Search (RLS)

Stochastic Hill-Climber - Greedy RLS

$\epsilon - greedy$ **RLS**

Decaying $\epsilon - greedy$ **RLS**

2.2.6 Few Pseudo-Boolean Problems

The *pseudo-Boolean* setting considers functions $f : \{0,1\}^n \to \mathbb{R}$ defined on binary strings of length n. Several benchmark functions are widely used to study and compare the behavior of randomized search heuristics, each encoding a different type of difficulty. [27; 12]

2.2.6.1 OneMax

The OneMax function counts the number of one-bits in a bit string:

$$\textsc{OneMax}(x) = \sum_{i=1}^{n} x_i, \qquad x \in \{0,1\}^n.$$

The unique global maximum is $x^* = 1^n$ with $\textsc{OneMax}(1^n) = n$. OneMax is the canonical baseline benchmark: the landscape is unimodal and every improving move increases the function value by exactly one, providing a clear gradient for local search methods. The expected runtime of the $(1+1)$ Evolutionary Algorithms (EA) on OneMax is $\Theta(n \log n)$. !CITE: Droste, Jansen, Wegener (2002)!

2.2.6.2 LeadingOnes

The LeadingOnes function counts the length of the maximal leading prefix of ones:

$$\textsc{LeadingOnes}(x) = \sum_{i=1}^{n} \prod_{j=1}^{i} x_j.$$

Equivalently, $\textsc{LeadingOnes}(x)$ equals the index of the first zero bit minus one, or n if no zero exists. The unique global optimum is again 1^n. Because progress requires the leading bit to be set before any subsequent bit can contribute, LeadingOnes is significantly harder than OneMax: the expected runtime of the $(1+1)$ EA is $\Theta(n^2)$. !CITE: Droste, Jansen, Wegener (2002)!

2.2.6.3 Jump

The Jump function with parameter $k \in \{1, \ldots, n\}$ coincides with OneMax on bit strings with at most $n-k$ ones and on the global optimum 1^n, but introduces a region of reduced fitness for strings with between $n-k+1$ and $n-1$ ones. This creates a "gap" of width k that algorithms relying on single-bit improvements cannot cross: the set of strings with exactly $n-k$ ones constitutes a local optimum

of standard local search, and reaching the global optimum 1^n requires flipping all k remaining zero bits simultaneously. For the $(1+1)$ EA with standard bit mutation at rate $1/n$, the probability of a specific k-bit flip is $\Theta(n^{-k})$, giving an expected runtime of $\Theta(n^k)$ on $\textsc{Jump}_k$.

Formally, $\textsc{Jump}_{k,n}\colon \{0,1\}^n \to \mathbb{R}$ is defined as

$$\textsc{Jump}_{k,n}(x) = \begin{cases} k + \sum_{i=1}^{n} x_i & \text{if } \sum_{i=1}^{n} x_i \leq n-k \text{ or } \sum_{i=1}^{n} x_i = n, \\ n - \sum_{i=1}^{n} x_i & \text{otherwise.} \end{cases}$$

Droste, Jansen, and Wegener [11] proved that the expected running time of the $(1+1)$ EA on $\textsc{Jump}_{k,n}$ is $\Theta(n^k + n\log n)$ for $k \in \{1,2,\ldots,n\}$, which simplifies to $\Theta(n^k)$ for $k \geq 2$.

2.2.6.4 BinaryValue[BinVal]

The BinaryValue (BinVal) function interprets a bit string as a binary integer by weighting bits exponentially by position:

$$\textsc{BinVal}(x) = \sum_{i=1}^{n} 2^{i-1} x_i.$$

The global optimum is 1^n with value $2^n - 1$. The exponential weighting means that the most significant bit contributes more than the combined value of all less significant bits, making BinVal highly structured.

2.2.6.5 Trap

2.2.6.6 Plateau

2.2.6.7 Hierarchical If-Only-If

2.2.6.8 MaxSAT

Maximum Satisfiability (MaxSAT) is the optimization variant of the Boolean satisfiability problem (SAT). Given a propositional formula in conjunctive normal form (CNF) !TODO: Cite or explain CNF! with m clauses over n Boolean variables, the goal is to find a truth assignment that maximizes the number of satisfied clauses. MaxSAT is NP-hard in general [19] and serves as an important

benchmark for combinatorial optimization algorithms.

2.2.6.9 Tunable Landscapes: NK-Landscapes

NK-landscapes [25] are a parametric family of pseudo-Boolean functions controlled by two integers: N (the number of variables) and K (the number of epistatic interactions per variable, $0 \leq K \leq N-1$). The fitness of a string $x \in \{0,1\}^N$ is defined as

$$f(x) = \frac{1}{N} \sum_{i=1}^{N} f_i\Big(x_i, x_{\pi(i,1)}, \ldots, x_{\pi(i,K)}\Big),$$

where each local contribution $f_i : \{0,1\}^{K+1} \to [0,1]$ is drawn independently and uniformly at random, and $\pi(i,\cdot)$ denotes the K epistatic neighbours of variable i.

When $K = 0$ the contributions are independent, producing a smooth, additively separable landscape analogous to OneMax. As K increases toward $N-1$, epistatic interactions intensify and the landscape becomes increasingly rugged, with the number of local optima growing in the limit. NK-landscapes thus provide a tunable model of ruggedness and are widely used to study algorithmic scalability and robustness across a spectrum of problem difficulties.

2.2.7 Traditional Performance Measures in BBO

2.3 Sequential and Online Optimization Frameworks

2.3.1 Exploration-Exploitation Trade-off

2.3.2 Multi-Armed Bandits

2.3.3 Online Optimization

2.3.4 Reinforcement Learning

2.4 Computational Regret

2.4.1 Definition of Regret

Let $f : \mathcal{X} \to \mathbb{R}$ be an objective function with global optimum value $f^* = \max_{x \in \mathcal{X}} f(x)$. An algorithm that queries points $x_1, x_2, \ldots, x_T$ incurs *regret* by querying suboptimal solutions. Regret quantifies the gap between the algorithm's performance and that of an oracle always querying the optimum [8]. !TODO: add more context with reference to MAB & rewards!

2.4.2 Types of Regret

2.4.2.1 Instantaneous Regret

The *instantaneous regret* at round t is

$$\rho_t = f^* - f(x_t),$$

capturing the suboptimality of the single query made at step t. A related variant tracks the *incumbent* (best-so-far) solution rather than the current query:

$$\rho_t^{\text{inc}} = f^* - f(\hat{x}_t), \quad \hat{x}_t = \arg\max_{s \le t} f(x_s).$$

Because $\hat{x}_t$ is non-decreasing in quality, ρ_t^{inc} is monotonically non-increasing over t, and is equivalent to simple regret evaluated at time t(2.4.2.3).

2.4.2.2 Cumulative Regret

Two cumulative regret variants arise naturally and serve different purposes.

The *Cumulative Regret (CR)* after T rounds is [8]

$$R_T = \sum_{t=1}^{T} \rho_t = \sum_{t=1}^{T} \Big(f^* - f(x_t) \Big).$$

This is the standard formulation from the online learning literature: it sums the instantaneous suboptimality of each individual query, regardless of what the algorithm has previously observed. Minimizing R_T corresponds to performing well *throughout* the optimization process, penalizing every suboptimal query equally.

The *incumbent cumulative regret* instead tracks the best-so-far process

$$B(t) := \max_{s \le t} f(x_s),$$

yielding

$$R_T^{\text{inc}} = \sum_{t=1}^{T} \rho_t^{\text{inc}} = \sum_{t=1}^{T} \Big(f^* - B(t) \Big).$$

Since $B(t)$ is non-decreasing in t, the per-step contributions ρ_t^{inc} are non-increasing, and $R_T^{\text{inc}} \le R_T$ for all T. The incumbent formulation is the appropriate notion when the practitioner retains the best solution found rather than only the most recent query, which is the standard convention for the randomized search heuristics studied in this project. All regret analyses in Chapter 4 use R_T^{inc} unless otherwise stated.

In continuous time, the incumbent cumulative regret is approximated via a left-hold (piecewise-constant) integral

$$R_T^{\text{inc}} \approx \sum_{t=1}^{T} \rho_{t-1}^{\text{inc}} \, \Delta t_t, \quad \Delta t_t = t - t_{t-1},$$

which accounts for unequal spacing between evaluations when function evaluation time is non-uniform. In all experiments reported here, evaluations are unit-cost and time is measured in discrete query counts, so $\Delta t_t = 1$ throughout and this reduces to the standard discrete sum.

2.4.2.3 Simple Regret

The *Simple Regret (SR)* after T rounds is [7]

$$r_T = f^* - f(\hat{x}_T),$$

where $\hat{x}_T = \arg\max_{t \le T} f(x_t)$ is the best solution encountered. Simple regret measures the quality of the *final output* and is the appropriate notion when the goal is to return a good solution rather than to perform well during the search. Note that $r_T = \rho_T^{\text{inc}}$: simple regret is precisely the incumbent instantaneous regret at the final round.

When comparing algorithms across multiple independent runs, the standard

scalar summary is the *expected simple regret*

$$\mathbb{E}[r_T] = \frac{1}{N}\sum_{i=1}^{N} r_T^{(i)},$$

where $r_T^{(i)}$ denotes the simple regret of the i-th run.

2.4.2.4 Normalized Regret

To enable comparison across problem instances with different objective scales, simple regret can be normalized by the range of attainable fitness values:

$$\tilde{r}_T = \frac{f^* - f(\hat{x}_T)}{f^* - f_{\text{worst}}} \in [0, 1],$$

where f_{worst} is the worst observed or theoretical fitness value. A value of 0 indicates an optimal solution was found; a value of 1 indicates no improvement over the worst case. When $f^* = f_{\text{worst}}$ (a degenerate constant landscape), the problem is trivially solved and normalized regret is defined as 0.

2.4.3 Regret in Black-Box Optimization

Regret provides a fine-grained complexity measure in the black-box setting: rather than simply asking whether an algorithm finds the optimum within T queries, regret tracks how suboptimal the queries are throughout the process. This is particularly relevant when every evaluation is costly and performing well before the budget is exhausted matters in practice.

2.4.4 Regret Bounds

2.4.5 Connections to Runtime Analysis

Chapter 3

Supplementary Software Design

3.1 Purpose of the Software

A supplementary experimental framework, `regret`, was developed to support the orchestration and analysis of experiments on pseudo-Boolean and combinatorial optimization problems. The framework was motivated by the absence of readily available tooling that combines (i) flexible, declarative experiment dispatch, (ii) trajectory-level data collection, and (iii) regret-specific analysis pipelines in a single cohesive system.

Existing platforms such as Google Vizier [14] and EvoX [22] address hyperparameter optimization and population-based search respectively, but neither provides native support for regret metrics, runtime profile analysis, or the configuration-first dispatch model required for this project's experimental protocol. `regret` was therefore purpose-built to fill this gap, providing a reproducible, extensible environment for studying simple, instantaneous, and cumulative regret across a controlled suite of algorithms and benchmark problems.

3.1.1 Preexisting software

Several established frameworks were considered prior to development. **Google Vizier** [14] provides a scalable black-box optimization service with a client–server architecture, but is oriented toward hyperparameter tuning rather than controlled algorithmic benchmarking, and does not expose trajectory-level data in a form amenable to regret analysis. **EvoX** [14] is a JAX-based evolutionary computation library offering GPU-accelerated population methods, but its abstraction

layer couples algorithm state to framework internals in ways that preclude the lightweight callback-driven trajectory recording required here. General-purpose scientific workflow systems such as Snakemake [29] and Luigi [13] offer pipeline orchestration but impose significant configuration overhead for the combinatorial (algorithm × problem × budget) sweeps central to this study.

3.1.2 Capabilities

The framework supports the full experimental life-cycle through a single Command-Line Interface (CLI), `run_experiment`, which accepts one or more YAML configuration files and dispatches the appropriate sub-command. Table 3.1 summarizes the available commands.

Table 3.1: CLI sub-commands exposed by the `run_experiment` entry point (equivalently `python -m regret`).

Command	Description
`validate`	Parse and validate one or more YAML configurations against the JSON Schema and semantic registry checks without executing any computation.
`plan`	Print a dry-run summary: problem/algorithm combinations, total runs, and evaluation budgets.
`run [--no-plot]`	Execute one or more experiment configurations; optionally suppress plot generation after completion.
`analyze`	Regenerate all plots from previously saved JSON results without re-running experiments.
`table [--format]`	Generate summary tables from existing results in `latex`, `csv`, or `markdown` format.

Experiment dispatch. All experiments are specified declaratively via YAML configuration files validated against a JSON Schema prior to execution. A configuration defines the suite name, evaluation budgets, run count, output mode, and lists of problem and algorithm specifications. Problems and algorithms are referenced by string keys resolved against internal registries, providing a clean separation between configuration and implementation.

Parallelised execution. When `parallel: true` is set, independent runs are dispatched via Python's `ProcessPoolExecutor`, with per-run random seeds fixed

to indices $0, 1, \ldots, r-1$ to ensure full reproducibility. Results are reordered by seed index after completion.

Flexible output modes. In `lite` mode, only summary statistics are persisted per run. In `full` mode, the complete evaluation trajectory (a sequence of `(evaluation_index, current_value, best_value)` tuples) is recorded and stored alongside aggregate statistics in structured JSON files.

Batched multi-configuration execution. All sub-commands accept multiple configuration files, enabling sequential execution of logically grouped experiment suites from a single invocation.

3.1.3 Design Decisions

The software was designed to be extensive & configurable for other developers and users to interact with quickly, with options for modularity when required. This was enabled by the adoption of configuration-first, job dispatch policy for independent and declarative experimentation. Problems and Algorithms to test against can easily be defined by extending abstract classes `Problem` & `Algorithm`, ensuring a common interface for consistent interactions.

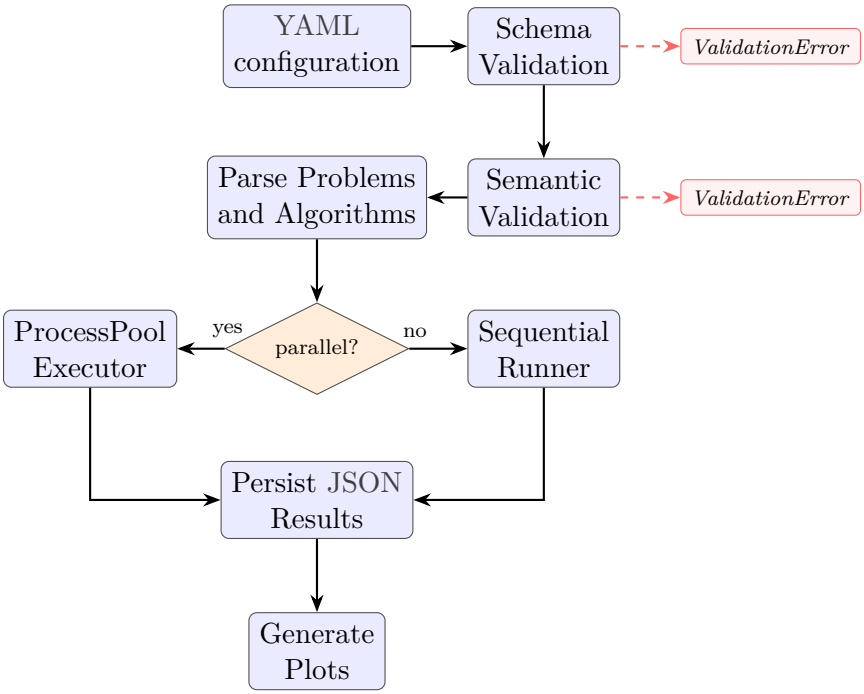
3.1.3.1 Dispatching framework

The CLI is implemented using **Typer** [34], which maps sub-commands to typed Python functions with automatic argument parsing. The orchestration layer (`regret.experiments.orchestration`) coordinates problem instantiation, algorithm dispatch, result persistence, and plot generation, delegating individual runs to `ExperimentRunner`. Parallel execution uses `ProcessPoolExecutor` from Python's standard `concurrent.futures` module; results are collected via `as_completed` and reordered by seed for deterministic output ordering. Figure 3.1 illustrates the full validation and dispatch pipeline.

3.1.3.2 UML Diagram

Figure 3.2 presents the core class structure of the framework. Abstract base classes `Problem` and `Algorithm` define the shared interface enforced across all concrete implementations. Concrete subclasses are registered by string key and

Figure 3.1: Validation and dispatch pipeline. YAML configuration files are validated structurally via JSON Schema and then semantically against class registries before execution begins. Any validation failure raises a `ValidationError` and halts the pipeline before computation begins.



resolved at runtime. `ExperimentRunner` orchestrates individual runs and delegates result persistence, while the `CoolingSchedule` protocol is resolved from YAML at parse time via a *registry*.

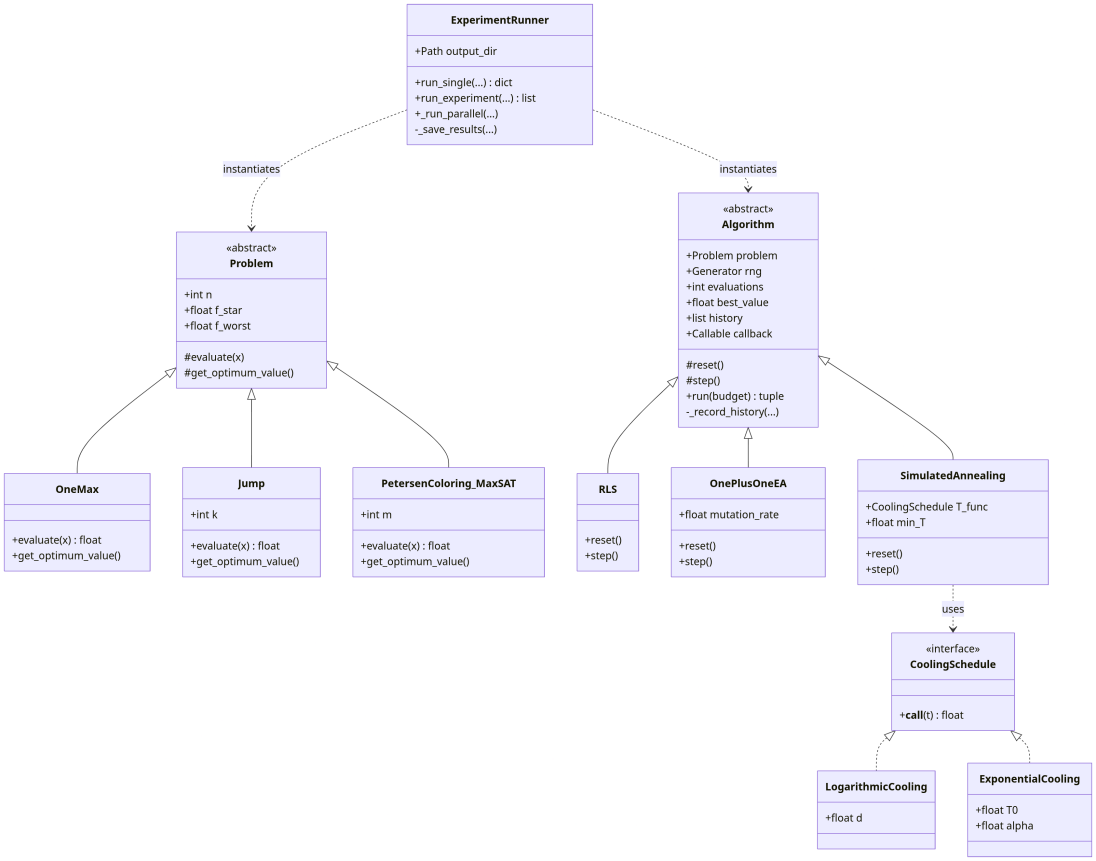
3.1.4 Limitations

The framework is currently constrained to **binary search spaces** $\{0,1\}^n$; continuous and integer-valued domains are not supported. **Multi-objective optimization** is not implemented. The NK landscape optimum computation uses exhaustive search, restricting its practical use to $n \leq 20$. Runtime profile analysis assumes integer-valued, unit-increment fitness functions for the exact tail-sum identity:

$$\mathbb{E}[\mathrm{CR}(T)] \;=\; \sum_{v=1}^{f^*} \sum_{t'=1}^{T} \Big[1 - P(\tau_v \leq t')\Big];$$

continuous fitness functions (e.g.`NK-Landscape`) yield approximate results and may exhibit minor discrepancies in the $\mathbb{E}[\mathrm{CR}(T)]$ verification plot.

Figure 3.2: Simplified UML class diagram of the `regret` framework. Solid triangle arrows (▷) denote inheritance; dashed arrows denote dependency or instantiation. Hashes '#' indicate abstract declarations. Several concrete subclasses (e.g. `MuPlusLambdaEA`, `RLSExploration`, `NKLandscape`, `LinearCooling`) are omitted for clarity.



3.1.5 Result Interpretations

All experiment output is written under a structured directory hierarchy. Raw JSON result files are stored at:

```
results/raw/<suite>/<problem>/<algorithm>/n<N>/b<B>.json
```

where `<suite>`, `<problem>`, and `<algorithm>` are normalized display names, `N` is the problem dimension, and `B` is the evaluation budget. Each file contains three top-level keys: `metadata` (algorithm, problem class, dimension, budget, run count, mode, and timestamp), `statistics` (mean, median, standard deviation, quartiles, and probability of optimum across all runs), and `results` (an ordered list of per-run dictionaries each containing regret, best value, optimality flag, evaluation count, seed, and the complete trajectory (in `full` mode)).

Generated figures are written under `results/figures/` following an analogous hierarchy:

```
results/figures/<suite>/<problem>/n<N>/<subdir>/<filename>.pdf
```

Plot subdirectories are configurable via `layout.structure` in the YAML configuration; the defaults are `aggregate/` for cross-budget plots, `history/` for trajectory and regret time-series plots, and `distributions/` for budget-specific boxplots and performance profiles. Runtime profile plots are written to a `profiles/` subdirectory within the raw results tree. All figures are saved as PDF at 300 DPI. Table 3.2 summarizes the full output organization.

Table 3.2: Output file organization for a representative experiment. "Normalization" replaces spaces, parentheses, and special characters with underscores or removes them.

Path pattern	**Contents**
Raw data (JSON)	
`results/raw/<suite>/<prob>/<algo>/n<N>/b<B>.json`	Per-budget result file: metadata, aggregate statistics, per-run regrets, and (in `full` mode) full evaluation trajectories.
`results/raw/<suite>/<prob>/profiles/n<N>/cr_profile_data_<algo>.csv`	Runtime profile CSV; columns: `evaluation`, `expected_cumulative_regret`, `mean_cumulative_regret`.
Figures (PDF, 300 DPI)	
`.../n<N>/aggregate/`	Cross-budget plots: regret curves, convergence probability, comparison heatmap.
`.../n<N>/history/`	Trajectory plots: best/current value, instantaneous and cumulative regret, TTFO distribution.
`.../n<N>/distributions/`	Budget-specific plots: regret boxplots and performance profiles.
`.../n<N>/profiles/`	Runtime profile plots: inverse profile surface/curves, $\mathbb{E}[\mathrm{CR}(T)]$ verification. Requires `suite.profile: true`.

Table 3.3: Plot types generated across experiments. "Profile required" denotes plots that require `suite.profile: true`

Plot	**Output filename pattern - `pdf` format**	**Profile required**
Regret curves	`regret_curves`	No
Convergence probability	`convergence_probability`	No
Comparison heatmap	`comparison_heatmap`	No
Regret boxplots	`regret_boxplot_b{budget}`	No
Performance profile	`performance_profile_b{budget}`	No
History (best-so-far)	`history_best`	No
Instantaneous regret	`history_regret_instantaneous`	No
Cumulative regret	`history_regret_cumulative`	No
TTFO distribution	`history_ttfo_markers`	No
Runtime profile surface	`inverse_runtime_profile_surface_{alg}`	Yes
Runtime profile curves	`inverse_runtime_profile_curves`	Yes
CR profile verification	`cr_profile_verification`	Yes

3.2 Testing strategy

The test suite follows a **Test-Driven Development** (TDD) discipline using a red–green workflow: tests were written prior to or alongside implementation and serve as executable specifications. Coverage is measured using `coverage` [5] with branch tracking enabled, and a minimum threshold of **50% branch coverage** is enforced in Continuous Integration (CI) via GitHub Actions. Coverage is also computed across Python 3.12, 3.13, and 3.14 in Continuous Integration (CI) for redundancy and support (the project must be compatible for `python` $\geq$ 3.12).

Test organization follows a selective weighting strategy: core algorithmic components (`test_algorithms.py`, `test_metrics.py`, `test_problems.py`) and integration paths (`test_integration.py`) receive dense coverage, while infrastructure components such as CLI wiring and plot dispatch are covered at a coarser granularity. Shared fixtures in `conftest.py` provide canonical problem instances and bitstrings used across test suites.

3.3 Project Dependencies

The project is built with `hatchling` [18] and managed with `uv` [2], a Rust-based package and environment manager that replaces the Python standard `pip/venv`. Runtime installation: `uv sync`; development installation: `uv sync --group dev`.

Table 3.4: Package dependencies for the `regret` framework (Python $\geq$ 3.12).

Package	**Min. version**	**Purpose**
Scientific computing		
`numpy` [17]	2.4.3	Array operations, random number generation
`scipy` [36]	1.16.3	Statistics, drift computations, profile integrals
`pandas` [31]	2.3.3	Results aggregation, tabulation, CSV export
Visualisation		
`matplotlib` [23]	3.10.8	All static plot generation
`ipympl`	0.10.0	Interactive Matplotlib in notebooks
`networkx` [16]	3.6.1	Graph problem instances (e.g. Petersen colouring)
Configuration and CLI		
`pyyaml` [35]	6.0.3	Experiment configuration parsing
`jsonschema` [32]	4.26.0	YAML schema validation
`typer` [34]	0.24.1	`run_experiment` CLI
`tqdm` [9]	4.67.1	Progress bars for parallel runs
Notebook and interactive demonstration		
`jupyter` [15]	1.1.1	Notebook server
`ipykernel`	7.2.0	Jupyter kernel support
`ipywidgets`	8.1.8	Notebook UI widgets
Development — linting (optional)		
`ruff` [1]	—	Fast Rust-based linter and formatter (E, W, F, I, B, UP rules)
Development — testing (optional)		
`pytest` [26]	—	Unit and integration test runner
`coverage` [5]	—	Branch coverage reporting ($\geq$50% enforced)
`tabulate`	—	Tabular output in test fixtures
Development — type checking (optional)		
`ty` [3]	—	Rust-based static type checker (Astral)
Development — documentation (optional)		
`zensical`	—	Documentation site generator
`mkdocstrings-python`		Auto-generated API reference from docstrings

Chapter 4

Experiments

4.1 Experiment Structure

All experiments share a common infrastructure defined in the YAML configuration files under `configs/experiments/` !TODO: Cite appendices!. Each configuration specifies four top-level blocks: `suite` (global settings), `problems` (benchmark functions), `algorithms` (optimisers), and `plotting` (output).

4.1.1 Repetitions and budget

Every algorithm–problem pair is evaluated over R independent runs (typically 25–100, chosen to keep Monte-Carlo variance !CITE! in $\mathbb{E}[\mathrm{CR}(T)]$ negligible), each run consuming up to a fixed evaluation budget $T_{\max}$ drawn from a per-suite budget list. A separate `budget_for_plots` value, calibrated to approximately twice the expected time-to-first-optimum ($\mathbb{E}[\mathrm{TTFO}]$) for the hardest algorithm on each problem, governs the time horizon displayed in trajectory plots.

4.1.2 Execution mode

All suites run in `full` mode with parallelism enabled. The `full` supports recording the evaluation time-series. It was trivially toggled for these experiments. Suites that compute inverse runtime profiles additionally set `profile: true`, which enables trajectory-level data collection required for the tail-sum verification and runtime-profile plots.

4.1.3 Visualisation and Table Strategy

The output of each suite consists of plots and summary tables generated from the stored JSON results. Plot types fall into four groups: aggregate, budget-specific distribution, trajectory, and runtime profile.

Aggregate plots. Three plots pool results across all budgets in the suite budget list. *Mean simple-regret curves* (`regret_curves`) plot $\widehat{\mathbb{E}}[f^* - f(x_{\text{best}})]$ against evaluation budget on a log–log axis with one shaded curve per algorithm; a steeper slope or earlier crossover indicates a faster-decaying algorithm. When all algorithms converge fully within the budget range this plot loses resolution, in which case the *convergence probability* plot (`convergence_probability`) is more informative: it shows the fraction of runs reaching regret below 10^{-9}, separating algorithms by the budget at which they first converge reliably. The *comparison heatmap* (`comparison_heatmap`) encodes $\log_{10}$(mean regret) as a colour over all algorithm–budget cells, providing a compact global overview but at lower precision than the regret curves.

Budget-specific distribution plots. These are generated once per problem at the budget specified by `budget_for_plots`. *Regret boxplots* (`regret_boxplots`) show the distribution of simple regret across runs with individual points overlaid, exposing skewness and run-to-run variability that means cannot: two algorithms can share the same mean while one converges reliably on most runs and fails completely on a few. *Performance profiles* (`performance_profile`) show the empirical CDF of simple regret on a log-scaled axis; a curve that rises steeply at low regret values indicates reliable near-optimal convergence, whereas a gradual rise indicates high variability. Both plots support optional pairwise statistical annotation (Mann–Whitney U and Cohen's d against a reference algorithm).

Trajectory plots. All trajectory plots are averaged across runs at `budget_for_plots`, with a standard-error band and optional TTFO markers at the mean time-to-first-optimum. The suite produces several complementary variants.

The *best-value history* (`history_best`) plots the monotonically non-decreasing incumbent fitness against evaluation time, providing an absolute fitness view with f^* as a horizontal reference.

Instantaneous regret is produced in two variants. With `track_incumbent:`

`false` (`regret_instantaneous`), it is the gap between f^* and the solution evaluated at each step, which fluctuates whenever an algorithm accepts a worsening move. With `track_incumbent: true` (`regret_instantaneous_best`), it is the gap from f^* to the incumbent, which is monotonically non-increasing and equivalent to simple regret measured at each t. For hill-climbing algorithms (RLS, $(1+1)$-EA) the two variants coincide because no worsening moves are accepted; for SA and RLSExploration they diverge during phases of active exploration. Both are plotted on a log-y axis. A flat non-zero floor in the incumbent variant on this axis is the primary diagnostic signature of saturation at a local optimum.

Cumulative regret (`regret_cumulative` and `regret_cumulative_best`) is the integral of the corresponding instantaneous regret series over time. It penalises slow convergence: an algorithm that reaches f^* late has accumulated all preceding regret regardless of final solution quality. The slope of the cumulative regret curve during the trapped phase grows linearly with time and is the key signal for barrier-crossing suites (05, 06, 09, 10); this slope is suppressed on a log axis, so cumulative regret is plotted on a linear y-axis throughout.

The *TTFO distribution* (`ttfo_distribution`) plots individual run-level times-to-first-optimum as jittered points per algorithm. This plot is conditioned on convergence: runs that did not reach the optimum contribute no point, so when $P(\text{opt}) \approx 0$ across the suite the scatter is sparse and adds little beyond the convergence probability plot.

Runtime profile plots. These require `suite.profile: true` and are produced only in suites 03, 08, and 12. The inverse runtime profile $P(\tau_v \leq T)$—the probability that fitness level v was first reached by evaluation T—is the common basis.

The *profile surface* (`inverse_runtime_profile_surface`) plots this probability as a heat map over (T, v) for each algorithm separately, with iso-probability contours overlaid. A front that stalls at some $v_{\text{local}} < f^*$ regardless of additional budget identifies the saturation level of the algorithm on that landscape. The *profile curves* (`inverse_runtime_profile_curves`) plot $P(\tau_v \leq T)$ versus T at four selected fitness levels ($f^* \times \{0.25, 0.5, 0.75, 0.95\}$), overlaying all algorithms on the same panel for direct comparison.

The *profile verification* plot (`cr_profile_verification`), enabled only in suite 08, overlays two independent estimates of $\mathbb{E}[\text{CR}(T)]$: the empirical mean

across runs, and the value derived via the tail-sum identity from the inverse profile. For integer-valued unit-increment fitness functions the identity is exact and any gap indicates a computational error; for the continuous-valued NK landscape (suite 12) a minor gap reflects finite-grid discretisation and not a bug.

Summary tables. Tables are exported per-suite in LaTeX, CSV, or Markdown. A *per-problem table* is produced at each budget for each problem, with one row per algorithm reporting mean, median, standard deviation, interquartile range, a 95% bootstrap confidence interval, $P(\text{opt})$, and run count, sorted by ascending mean regret. The mean–median gap is informative when distributions are skewed by a minority of non-converging runs. An *aggregate cross-problem table* summarises each algorithm's across-problem mean regret, mean $P(\text{opt})$, and the count of problems on which it achieved the lowest mean regret among all algorithms in the suite.

4.1.4 Experiment numbering

Suites are numbered 01–14. Suites 01–02 establish the baseline algorithm-family comparison. Suites 03–07 and 09–14 form the exploratory analyses. Suite 08 is a dedicated verification suite for the tail-sum identity. Suites 03 and 12 additionally enable runtime profiling to study saturation phenomena on NK landscapes alongside the verification suite.

4.2 Runtime Profiles

4.2.1 Motivation

The predominant performance measure in the runtime analysis of EAs is the *expected optimization time*: the expected number of function evaluations until an optimal solution is queried for the first time [11; 10]. While this measure has proved immensely useful for ranking algorithms and establishing complexity hierarchies, it carries a significant limitation for practice. In almost all realistic settings, a user cannot detect when the algorithm has found an optimum. Even if this were possible, the number of evaluations required to guarantee optimality may be prohibitively large [33]. What the practitioner actually observes is the evolution of solution quality over the *entire* course of the run.

Two complementary frameworks have been proposed to address this limitation. The first, from Jansen and Zarges [24], adopts a *fixed-budget perspective*: fix the evaluation budget T in advance and characterize the expected quality of the best solution found within that budget. The second, introduced by Pinto and Doerr [33], proposes *runtime profiles*: rather than reporting only the expected time to reach the global optimum, report the expected time to reach each intermediate fitness level. These two views are orthogonal; one fixes time and studies expected solution quality, the other fixes solution quality and studies expected time. However, both provide richer information than the single expected optimization time, as supported by Pinto and Doerr:

> Our intention is rather to highlight to researchers working on theoretical aspects in evolutionary computation that, beyond being possibly more relevant for practitioners, explicitly reporting runtime profiles can give quite interesting insights into the performance of EAs. ([33], p. 19)

4.2.2 Fixed-Budget Analysis

In *fixed-budget analysis* [24], the evaluation budget T is fixed in advance and the quantity of interest is the distribution of solution quality achieved within that budget. Formally, let $f_T^*(\mathcal{A})$ denote the best objective value achieved by algorithm $\mathcal{A}$ within T evaluations on objective f. Fixed-budget analysis characterizes $\mathbb{E}[f_T^*(\mathcal{A})]$ as a function of T [24].

Fixed-budget analysis is particularly relevant in practice, where resources are finite and an algorithm must return the best solution found when the budget is exhausted, regardless of whether the global optimum has been identified [24]. It connects naturally to the simple regret framework: minimizing simple regret over a fixed budget T is equivalent to maximizing $\mathbb{E}[f_T^*(\mathcal{A})]$. Comparing two algorithms under a fixed budget may yield different conclusions than comparing their expected runtimes, since an algorithm with a larger expected optimization time may still produce better intermediate solutions within a given budget [24].

4.2.3 Variants

4.2.3.1 Expectation-Based Runtime Profiles

Pinto and Doerr [33] propose that runtime results for evolutionary algorithms should include not only the expected time to reach the global optimum but also the expected time needed to reach each intermediate fitness level. Concretely, for a fitness function $f : \{0,1\}^n \to \mathbb{R}$ with global optimum $f^* = \max_x f(x)$, and for each target level v, the *first hitting time* of level v is

$$\tau_v := \min\Big\{t \in \mathbb{N} : f(x_t) \geq v\Big\}, \tag{4.1}$$

where $x_1, x_2, \ldots$ is the sequence of query points produced by the algorithm [11]. The *expectation-based runtime profile* of an algorithm is the collection of expected first hitting times $\Big(\mathbb{E}[\tau_v]\Big)_v$ across all relevant target levels [33].

When canonical fitness levels exist, such as for OneMax or LeadingOnes, these provide natural targets; for other functions, intermediate values identified during the analysis or obtained by linear interpolation of the minimal and maximal fitness values may be used [33]. Crucially, Pinto and Doerr emphasize that this framework does not require new algorithms or new proof techniques, all bounds can be derived from existing runtime results and are often already implicit in the proofs [33]. The proposal is therefore one of *presentation*: reporting richer, more informative performance summaries that better reflect the anytime behavior of an algorithm.

The practical value of this richer view is illustrated by the behavior of (1+1) EA variants on LeadingOnes: reporting only the expected optimization time suggests RLS is superior, yet the runtime profile reveals that (1+1) $\text{EA}_{>0}$ (see !TODO: ref (1+1)-EA_>=0 variant!) reaches all intermediate fitness levels $i \leq 7{,}980$ faster than RLS for problem size $n = 10{,}000$, with RLS only pulling ahead in the final portion of the run [33]. Such crossover behavior is invisible to the single expected optimization time and is directly relevant to the design of adaptive parameter and operator selection schemes [33].

4.2.3.2 Distributional Runtime Profiles

The expectation-based runtime profile of [33] summarizes the distribution of τ_v by its mean. A finer characterization, and the appropriate quantity for connecting

runtime profiles to regret-based measures, is the *distributional runtime profile*

$$\mathcal{P}(v,t) := \mathbb{P}(\tau_v > t), \tag{4.2}$$

which records, for each target level v and time step t, the probability that level v has not yet been reached by time t. The expectation-based profile is recovered by the standard tail-sum formula for non-negative integer-valued random variables [10]:

$$\mathbb{E}[\tau_v] = \sum_{t=0}^{\infty} \mathbb{P}(\tau_v > t). \tag{4.3}$$

The distributional profile is the more primitive object from which expectation-based statements follow, and it constitutes the link between runtime profiles and the regret-based framework developed below.

Remark 2 (Naming convention)**.** *The term* runtime profile *appears in two distinct senses in the literature and in this work. Pinto and Doerr [33] use it to denote the* expectation-based *runtime profile* $\left(\mathbb{E}[\tau_v]\right)_v$*, which summarizes per-level performance by expected hitting times. The present work additionally employs the strictly finer* distributional *runtime profile* $\mathcal{P}(v,t) = \mathbb{P}(\tau_v > t)$*, from which the expectation-based profile is recovered via the tail-sum formula (Equation 4.3). To avoid ambiguity, the two notions are referred to explicitly whenever both are in scope. Hereafter, and in all practical experimentation in this report,* runtime profile *refers to the distributional variant* $\mathcal{P}(v,t)$ *unless otherwise stated.*

4.2.4 Connection to Cumulative Regret

One of the discussions suggested by Pinto and Doerr in their *Future Works* section is highlighted:

> We feel that there is a need for combined measures that are capable of describing the anytime behavior of an EA['Evolutionary Algorithms']. Regret-based measure as used in machine learning could be a key here, but we haven't been able so far to identify a fully satisfying measure. ([33], p. 22)

As far as we know, establishing such a relationship remains an open question. This consolidated the following proposal for exploring the established concepts of *Cumulative Regret (CR)* and *Runtime Profiles*.

The relationship between runtime profiles and CR can be made precise through a formal setup. A randomized black-box algorithm produces an adaptive sequence of query points $x_1, x_2, \ldots \in \{0,1\}^n$. Define the *best-so-far process*

$$B(t) := \max_{s \le t} f(x_s),$$

which records the best objective value seen up to time t. Since $B(t)$ is non-decreasing in t, the event $\{B(t) < v\}$ is equivalent to $\{\tau_v > t\}$ (the level v has not yet been hit). The *CR* with respect to the global optimum f^* over a budget of T evaluations is [8]

$$\mathrm{CR}(T) := \sum_{t=1}^{T} \Big(f^* - B(t) \Big).$$

The proof of the main theorem relies on two elementary lemmas.

Lemma 1 (Tail-sum representation for the gap)**.** *For any integer b with $0 \le b \le f^*$,*

$$f^* - b = \sum_{v=1}^{f^*} \mathbf{1}[b < v].$$

Proof. The right-hand side counts the integers $v \in \{1, \ldots, f^*\}$ that strictly exceed b, namely the set $\{b+1, \ldots, f^*\}$, which has cardinality $f^* - b$. □

Lemma 2 (Indicator equivalence)**.** *For all $v \in \{1, \ldots, f^*\}$ and $t \in \mathbb{N}$,*

$$\mathbf{1}[B(t) < v] = \mathbf{1}[\tau_v > t].$$

Proof. Since $B(t)$ is non-decreasing in t, the event $\{B(t) \ge v\}$ is equivalent to $\{\tau_v \le t\}$. Taking complements gives $\{B(t) < v\} = \{\tau_v > t\}$, so the two indicators agree on every sample path. □

The following identity connects *expected* cumulative regret to the distributional runtime profile.

Theorem 3 (Tail-Sum Regret Representation)**.** *For any randomized black-box algorithm operating on f with finite budget $T \in \mathbb{N}$,*

$$\mathbb{E}[\mathrm{CR}(T)] = \sum_{v=1}^{f^*} \sum_{t=1}^{T} \mathbb{P}(\tau_v > t).$$

Proof. For each t, Lemma 1 applied to $b = B(t)$ gives

$$f^* - B(t) \;=\; \sum_{v=1}^{f^*} \mathbf{1}[B(t) < v].$$

Summing over $t = 1, \ldots, T$ yields

$$\mathrm{CR}(T) \;=\; \sum_{t=1}^{T} \Big(f^* - B(t) \Big) \;=\; \sum_{t=1}^{T} \sum_{v=1}^{f^*} \mathbf{1}[B(t) < v].$$

By Lemma 2, $\mathbf{1}[B(t) < v] = \mathbf{1}[\tau_v > t]$ on every sample path, so

$$\mathrm{CR}(T) \;=\; \sum_{t=1}^{T} \sum_{v=1}^{f^*} \mathbf{1}[\tau_v > t].$$

Taking expectations and applying linearity of expectation together with $\mathbb{E}[\mathbf{1}[\tau_v > t]] = \mathbb{P}(\tau_v > t)$ gives

$$\mathbb{E}[\mathrm{CR}(T)] \;=\; \sum_{t=1}^{T} \sum_{v=1}^{f^*} \mathbb{P}(\tau_v > t).$$

Since both sums are finite, their order may be exchanged, yielding the stated result. □

Theorem 3 expresses expected cumulative regret as a double sum of the distributional runtime profile: $\mathbb{E}[\mathrm{CR}(T)]$ is the total expected time the algorithm spends below each fitness level v, summed over all levels. An algorithm with a uniformly smaller runtime profile (one that reaches each fitness level earlier in expectation) incurs lower cumulative regret.

Taking the infinite-horizon limit, and applying the tail-sum identity for $\mathbb{E}[\tau_v]$ along with the monotone convergence theorem, yields the following asymptotic connection.

Corollary 4 (Asymptotic Regret and Expected Hitting Times)**.** *Suppose that τ_v is almost surely finite with finite expectation for each $v \in \{1, \ldots, f^*\}$. Then*

$$\lim_{T \to \infty} \mathbb{E}[\mathrm{CR}(T)] \;=\; \sum_{v=1}^{f^*} \mathbb{E}[\tau_v] \;-\; f^*.$$

Proof. **Step 1: Tail-sum identity for $\mathbb{E}[\tau_v]$.** Since $\tau_v \in \mathbb{N}_{\geq 1}$ by the convention that $t = 1$ is the first oracle call and initialization is excluded from the query

budget, we have $\mathbb{P}(\tau_v > 0) = 1$. The standard tail-sum formula for a positive integer-valued random variable then gives

$$\mathbb{E}[\tau_v] \;=\; \sum_{t=0}^{\infty} \mathbb{P}(\tau_v > t) \;=\; 1 + \sum_{t=1}^{\infty} \mathbb{P}(\tau_v > t),$$

hence $\sum_{t=1}^{\infty} \mathbb{P}(\tau_v > t) = \mathbb{E}[\tau_v] - 1$.

Step 2: Passing to the limit in Theorem 3. For each fixed v, the partial sums $\sum_{t=1}^{T} \mathbb{P}(\tau_v > t)$ are non-decreasing in T and non-negative, so the monotone convergence theorem gives

$$\lim_{T\to\infty} \sum_{t=1}^{T} \mathbb{P}(\tau_v > t) \;=\; \sum_{t=1}^{\infty} \mathbb{P}(\tau_v > t) \;=\; \mathbb{E}[\tau_v] - 1.$$

Step 3: Summing over v. Applying Step 2 to each $v = 1, \ldots, f^*$ and invoking Theorem 3,

$$\lim_{T\to\infty} \mathbb{E}[\mathrm{CR}(T)] \;=\; \sum_{v=1}^{f^*} \Big(\mathbb{E}[\tau_v] - 1\Big) \;=\; \sum_{v=1}^{f^*} \mathbb{E}[\tau_v] \;-\; f^*. \qquad \square$$

Corollary 4 provides a unifying perspective: the long-run expected cumulative regret decomposes as the sum of the expected first hitting times over all fitness levels, minus a constant equal to f^* (arising from the $t = 0$ term in the tail-sum identity). Algorithms that reach all intermediate fitness levels quickly are therefore precisely those that incur low long-run cumulative regret. This formalizes the intuition, advanced in [33], that an algorithm making steady progress through the fitness landscape is preferable to one that stagnates at low fitness values even if both ultimately find the optimum in the same expected time.

The identity also reveals that the regret-based measures studied in the online learning literature [8] and the runtime profile framework of [33] are not independent approaches to measuring algorithm performance but are two views of the same underlying quantity, connected through the distributional runtime profile $\mathcal{P}(v, t)$.

4.2.5 Practical Relevance

The runtime profile framework addresses a fundamental limitation of the single expected optimization time as a performance measure: it cannot distinguish

between algorithms that make steady progress and those that stagnate before eventually finding the optimum. In practice, most applications operate under a finite evaluation budget, and intermediate solution quality at every point in the run matters [24; 33].

Pinto and Doerr [33] note that runtime profiles are particularly valuable for the design of non-static parameter and operator selection strategies in EAs (such as adaptive mutation rates, crossover probabilities, and selection pressure) since such strategies must make decisions based on current solution quality rather than on the expected optimization time alone. The identification of *crossover fitness levels* (where one algorithm overtakes another in the runtime profile) provides actionable guidance for when to switch between strategies.

Furthermore, as established in Theorem 3, any collection of upper bounds on $\mathbb{E}[\tau_v]$ for all levels v simultaneously yields an upper bound on expected cumulative regret, and conversely, lower bounds on regret propagate to lower bounds on a weighted aggregate of first hitting times. Cumulative regret thus provides a single scalar summary of the entire runtime profile, complementing the per-level view that the profile itself offers.

Pinto and Doerr [33] explicitly identify regret-based measures from machine learning as a promising direction for capturing the anytime behavior of an EA within a unified complexity framework, while noting that identifying a fully satisfying combined measure remains an open problem. The identity in Theorem 3 and Corollary 4 can be seen as concrete progress toward that goal.

4.3 Baseline Suites

4.3.1 Suite 01: Algorithm-Family Comparison

Suite 01 compares four algorithm families across six pseudo-boolean problems of increasing difficulty. The algorithms are: RLS, RLSExploration, the $(1+1)$-EA, and a single SA variant with logarithmic cooling $d = 2.0$. Problems are OneMax, LeadingOnes, TwoMax, Jump-3, Trap-10, and BinVal, all with $n = 100$ except BinVal ($n = 50$). Budgets range from 200 to 10 000 across 50 independent runs.

BinVal is included for its exponential fitness structure but is excluded from profile analysis: with $f^* = 2^{50} - 1$, constructing an integer fitness-level grid is

computationally infeasible. Profile plots are disabled suite-wide. Pairwise statistical annotations are enabled on boxplots, performance profiles, and Time to First Optimum (TTFO) distributions, with RLS as the reference algorithm.

4.3.2 Suite 02: Extended Pseudo-Boolean Benchmark

Suite 02 extends the baseline to nine problems, adding Jump-5, Trap-5, Plateau-10, HIFF-64, and a combinatorial instance (Petersen-3-Color). The algorithm set is identical to suite 01 (the same four families), with problem-specific hyperparameter overrides for RLSExploration ($\varepsilon = 0.05$, no decay on Jump-3, Jump-5, Trap-5) and SA-Log-d_2.0 ($d = 6.0$ on Trap-5 and Jump-5). Budgets range from 500 to 20 000 over 50 runs. Profile analysis is disabled.

4.4 Exploratory Analysis of Regret and Runtime

The exploratory suites address four thematic research questions:

1. Algorithm-family ordering across problem classes.
2. The effect of landscape barrier structure and mutation rate on regret trajectory shape.
3. The interaction between SA cooling schedules and landscape ruggedness.
4. Regret and convergence scaling with problem size and population structure.

Chapter 5

Evaluation & Discussion of Results

5.1 PLACEHOLDER

5.2 PLACEHOLDER

5.3 PLACEHOLDER

Chapter 6

Conclusion & Future Work

6.1 Conclusion

6.2 Future Work

Bibliography

[1] Astral. Ruff: An extremely fast Python linter and code formatter, 2022. Accessed: 2026.

[2] Astral. uv: An extremely fast Python package and project manager, 2024. Accessed: 2026.

[3] Astral. ty: A fast Python type checker and language server, 2025. Accessed: 2026.

[4] Charles Audet and Warren Hare. *Derivative-free and blackbox optimization*, volume 1. Springer, 2017.

[5] Ned Batchelder and contributors. coverage.py: Code coverage measurement for Python, 2009. Accessed: 2026.

[6] Dimitri Bertsekas. *Convex optimization theory*, volume 1. Athena Scientific, 2009.

[7] Sébastien Bubeck, Rémi Munos, and Gilles Stoltz. Pure exploration in finitely-armed and continuous-armed bandits. *Theoretical Computer Science*, 412(19):1832–1852, 2011.

[8] Nicolo Cesa-Bianchi and Gábor Lugosi. *Prediction, learning, and games*. Cambridge university press, 2006.

[9] Casper da Costa-Luis, Stephen Karl Larroque, Kyle Altendorf, Hadrien Mary, richardsheridan, Mikhail Korobov, Noam Raphael, Ivan Ivanov, Marcel Bargull, Nishant Rodrigues, Shawn, Mikhail Dektiarev, Michał Górny, mjstevens777, Matthew D. Pagel, Martin Zugnoni, CrazyPython, Charles Newey, Antony Lee, pgajdos, Todd, Staffan Malmgren, redbug312, Orivej

Desh, Nikolai Nechaev, Mike Boyle, Max Nordlund, MapleCCC, Jack McCracken, and Hugo van Kemenade. tqdm: A fast, extensible progress bar for python and cli, February 2026.

[10] Benjamin Doerr and Frank Neumann. Theory of evolutionary computation. *Recent Developments in Discrete Optimization. Springer, Cham*, 2020.

[11] Stefan Droste, Thomas Jansen, and Ingo Wegener. On the analysis of the (1+ 1) evolutionary algorithm. *Theoretical Computer Science*, 276(1-2):51–81, 2002.

[12] Stefan Droste, Thomas Jansen, and Ingo Wegener. Upper and lower bounds for randomized search heuristics in black-box optimization. *Theory of computing systems*, 39(4):525–544, 2006.

[13] Elias Freider Erik Bernhardsson. Luigi, 2012. Accessed: 2026.

[14] Daniel Golovin, Benjamin Solnik, Subhodeep Moitra, Greg Kochanski, John Karro, and D. Sculley. Google vizier: A service for black-box optimization. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Halifax, NS, Canada, August 13 - 17, 2017*, pages 1487–1495. ACM, 2017.

[15] Brian E. Granger and Fernando Pérez. Jupyter: Thinking and storytelling with code and data. *Computing in Science & Engineering*, 23(2):7–14, 2021.

[16] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. NetworkX, 2008. Accessed: 2026.

[17] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.

[18] Hatch Developers. Hatchling: The build backend for the Hatch project manager, 2022. Accessed: 2026.

[19] Holger H. Hoos, Kevin Smyth, and Thomas Stützle. Search space features underlying the performance of stochastic local search algorithms for MAX-SAT. In Xin Yao, Edmund K. Burke, José A. Lozano, Jim Smith, Juan Julián Merelo-Guervós, John A. Bullinaria, Jonathan E. Rowe, Peter Tiňo, Ata Kabán, and Hans-Paul Schwefel, editors, *Parallel Problem Solving from Nature - PPSN VIII*, volume 3242, pages 51–60. Springer Berlin Heidelberg, 2004. Series Title: Lecture Notes in Computer Science.

[20] Holger H. Hoos and Thomas Stützle. 1 - introduction. In Holger H. Hoos and Thomas Stützle, editors, *Stochastic Local Search*, The Morgan Kaufmann Series in Artificial Intelligence, pages 13–59. Morgan Kaufmann, San Francisco, 2005.

[21] Holger H. Hoos and Thomas Stützle. 2 - sls methods. In Holger H. Hoos and Thomas Stützle, editors, *Stochastic Local Search*, The Morgan Kaufmann Series in Artificial Intelligence, pages 61–112. Morgan Kaufmann, San Francisco, 2005.

[22] Beichen Huang, Ran Cheng, Zhuozhao Li, Yaochu Jin, and Kay Chen Tan. EvoX: A Distributed GPU-accelerated Framework for Scalable Evolutionary Computation. *IEEE Transactions on Evolutionary Computation*, 2024.

[23] John D Hunter. Matplotlib: A 2d graphics environment. *Computing in science & engineering*, 9(3):90–95, 2007.

[24] Thomas Jansen and Christine Zarges. Performance analysis of randomised search heuristics operating with a fixed budget. *Theoretical Computer Science*, 545:39–58, 2014.

[25] Stuart Kauffman and Simon Levin. Towards a general theory of adaptive walks on rugged landscapes. *Journal of theoretical Biology*, 128(1):11–45, 1987.

[26] Holger Krekel, Bruno Oliveira, Ronny Pfannschmidt, Floris Bruynooghe, Brianna Laugher, and Freya Bruhin. pytest x.y, 2004. Contributors: Holger Krekel and Bruno Oliveira and Ronny Pfannschmidt and Floris Bruynooghe and Brianna Laugher and Freya Bruhin and others.

[27] Johannes Lengler. Drift analysis. In *Theory of evolutionary computation: Recent developments in discrete optimization*, pages 89–131. Springer, 2019.

[28] Sijia Liu, Pin-Yu Chen, Bhavya Kailkhura, Gaoyuan Zhang, Alfred O. Hero, and Pramod K. Varshney. A Primer on Zeroth-Order Optimization in Signal Processing and Machine Learning: Principals, Recent Advances, and Applications. *IEEE Signal Processing Magazine*, 37(5):43–54, September 2020.

[29] F Mölder, KP Jablonski, B Letcher, MB Hall, PC van Dyken, CH Tomkins-Tinch, V Sochat, J Forster, FG Vieira, C Meesters, S Lee, SO Twardziok, A Kanitz, J VanCampen, V Malladi, A Wilm, M Holtgrewe, S Rahmann, S Nahnsen, and J Köster. Sustainable data analysis with snakemake [version 3; peer review: 2 approved]. *F1000Research*, 10(33), 2025.

[30] Jorge Nocedal and Stephen J Wright. *Numerical optimization*. Springer, 2006.

[31] The pandas development team. pandas-dev/pandas: Pandas, February 2020.

[32] Julian Perelman and contributors. jsonschema, 2012. Accessed: 2026.

[33] Eduardo Carvalho Pinto and Carola Doerr. Towards a more practice-aware runtime analysis of evolutionary algorithms. *arXiv preprint arXiv:1812.00493*, 2018.

[34] Sebastián Ramírez. Typer, 2019. Accessed: 2026.

[35] Kirill Sirenkov and contributors. PyYAML, 2006. Accessed: 2026.

[36] Pauli Virtanen, Ralf Gommers, Travis E Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. Scipy 1.0: fundamental algorithms for scientific computing in python. *Nature methods*, 17(3):261–272, 2020.